



# Utilizing phonetic similarity for cross-source and cross-language toponym matching: a benchmark and prototype

Tomer Sagi<sup>1</sup> · Moran Zaga<sup>2</sup> · Sinai Rusinek<sup>2</sup> · Marcell R. Fekete<sup>1</sup> · Johannes Bjerva<sup>1</sup> · Katja Hose<sup>1,3</sup>

Accepted: 21 January 2025 / Published online: 26 February 2025  
© The Author(s) 2025

## Abstract

The writings of one ancient civilization often overlap in time and space with others. Many of these sources comprise unstructured text in ancient languages, causing scholars studying these civilizations to be siloed, often relying on sources in specific languages. Most recent efforts to extract structured information from historical scripts into place (toponym) and people databases (prospographies) have followed this pattern, focusing on one civilization and selected sources. The path to creating a common database runs through aligning names or toponyms between sources from disparate languages utilizing different scripts. Existing multi-lingual orthographic (string-based) comparison often relies on transliteration to a common script (Latin/English). Transliteration often creates multiple options and even more confusion. However, when integrating sources that overlap in space and time, the languages often share a common phonetic background. This commonality may prove beneficial. In this work, we present a benchmark for comparing toponyms from two linguistically and culturally related languages, namely Hebrew and Arabic. We provide a benchmark comprised of a set of dataset pairs created from historical sources written in Medieval variants of these languages, later historical Gazetteers and a modern dataset curated from Wikidata. We empirically evaluate several toponym comparison approaches over the benchmark: transliteration to a common script, direct transliteration, and phonetic comparison using a common phonetic representation. We discuss the results and the limitations of the various methods and outline future work.

**Keywords** Toponym matching · Multi-lingual · Grapheme to phoneme · Transliteration

## 1 Introduction

Studying ancient civilizations requires specialization in the language, cultural background, and even specific scripts used in the period and geography under study. Scholars must learn how to read texts in various ancient, often derelict languages, written in scripts that are not only diverse but also archaic. This linguistic diversity poses a significant challenge, as it leads to a compartmentalization of academic efforts, with researchers tending to specialize in sources from specific languages or scripts. This specialization often results in silos of knowledge that are difficult to cross-reference and integrate, thus overlooking the interconnected nature of the studied civilizations, especially those that existed contemporaneously and geographically proximate to each other. Thus, the ability of scholars from disparate languages and cultural focuses to collaborate over shared data is severely limited. A scholar in Arabic literature of the Middle Ages will be severely challenged when attempting to incorporate information and even raw data from repositories in Hebrew, even though they are from the same period and may substantially overlap in the places mentioned and events referenced. Recently, there have been several cross-lingual reconciliation efforts to create shared repositories for toponyms (place names) and names of people, such as the Pleiades project (Pleiades Team, 2024), the World Historical Gazetteer (Grossner & Mostern, 2021), and the Historical Index of the Medieval Middle East (Carlson, 2024). However, many of these attempts are hampered by the lack of robust automated tools for aligning place names across languages and benchmarks to test these over. Moreover, machine-learning-based approaches are gaining much traction and impressive results over modern languages with a wide base for training and testing resources available to them. However, no such capabilities are available for ancient languages, such as ancient Hebrew and ancient Arabic.

A critical step in overcoming these challenges lies in aligning and comparing names and places (toponyms) across different sources and languages. This task is particularly challenging when dealing with languages that utilize distinct scripts. The conventional method of multi-lingual orthographic comparison often relies on transliterating these texts into a common script, typically Latin or English. However, this approach has its drawbacks, as transliteration can lead to multiple interpretations and further confusion, especially when dealing with ancient variants of languages.

The following examples of the Hebrew and Arabic names of Damascus, Tyre and Tiberias in Table 1 exhibit the problematics in matching Semitic names through a non-Semitic intermediary—whether translation or transliteration. The Hebrew and Arabic forms are ostensibly more similar to each other than the Latin/English language name. The mere transliteration into Latin script also adds distance, which

**Table 1** Comparison of three toponyms in Hebrew and Arabic scripts and their transliteration to Latin script

| Original script             | Transliteration to Latin script |
|-----------------------------|---------------------------------|
| Hebrew: דמשק Arabic: دمشق   | Dameseq ⇔ Dimashq               |
| Hebrew: צור Arabic: صور     | Sör ⇔ Sūr                       |
| Hebrew: טבריה Arabic: طبرية | Tveria ⇔ Ṭabariyā               |

would not be there if the transliteration was direct. The main cause is the explicit writing, in the Latin transliteration, of vowels that are implicit in the Semitic names and differ between the two source languages.

In light of these challenges, our research focuses on a novel approach to comparing toponyms from two linguistically and culturally related languages: Hebrew and Arabic. Both languages, with their Semitic roots, share a common phonetic background despite their different scripts and historical evolutions. We posit that this phonetic commonality can be leveraged to develop more effective methods for toponym comparison.

Our motivating use case is that of a humanities scholar, such as a historian or a geographer, which normally focuses their research on a specific culture and period in history and is interested in enriching their knowledge and completing the narratives they are examining using other sources describing the same places, perhaps from other cultures and languages. Thus, our users would typically have a list of places extracted from a literary source such as a travelogue (Benjamin, n.d) and would be interested in matching them with their mentions in other sources. For example, Benjamin of Tudela (Benjamin, n.d), mentions a legend about the city of Baalbec, which was said to be built by King Solomon for the daughter of the Egyptian pharaoh. By matching this mention of Baalbec with its mention in a contemporary Islamic gazetteer Yāqūt (1977), the scholar can find that a similar legend was told about Baalbec. Interestingly enough, in this version, the king built it in honor of the Queen of Sheba. Figure 1 presents a screenshot of the reconciliation screen in the MEHDIE tool that allows users to review the results of such a matching process.

In this work, we present our initial foray into the complex problem of cross-lingual toponym matching. We present a new benchmark comprised of place pairs from two different geographical sources, originally in two different ancient languages (Hebrew and Arabic), overlapping in space and time. We investigate the viability of two approaches towards cross-lingual toponym matching. The first approach utilizes transliteration rules between the two languages to perform a direct orthographic

The screenshot displays the 'Reconciliation Review' interface of the MEHDIE tool. On the left, a sidebar shows the dataset 'TudelaNotes' and a list of items, with 'בעל ביק' (Baalbek) selected. The main panel shows the details of the selected item, including its MEHDIE ID (109137), title, and various links. On the right, a detailed view of the match is shown, including the name 'بعلبك' (Baalbek), the explanation 'Fuzzy name variant match over threshold 0.95 (confidence: 1.0)', and the source 'md (dlg05108)'. The interface also includes a 'Save' button and a 'Record 5 of 54' indicator.

**Fig. 1** Motivating use-case, matched mentions of the city of Baalbek from two contemporary sources in Ancient Hebrew and Ancient Arabic

comparison in one or the other language. The second approach maps both scripts into a phonetic representation, specifically the International Phonetic Alphabet (IPA), which allows for a comparison based on phonology (speech sounds) rather than orthography. Our evaluation employs previously suggested phonetic distance metrics (Mortensen et al., 2016) that have not yet been evaluated in a cross-lingual similarity task.

We empirically evaluate these approaches both on our proposed benchmark and on a synthetic benchmark generated by mining the labels of toponyms in modern-day versions of these languages, i.e., modern Hebrew and modern standard Arabic from wikidata. We thereby make the following contributions: (1) Direct Transliteration method between Arabic and Hebrew; (2) Rule-enhanced grapheme to phoneme model for Hebrew and Arabic Toponyms; (3) Wikidata-based curated toponym comparison benchmark; (4) Set of matched toponym datasets from historical sources in related Semitic languages; (5) Phonetic-based toponym comparison method; and (6) Empirical evaluation of three basic approaches to toponym matches, namely—direct transliteration, transliteration to a common script, and phonetic comparison using IPA representations. (7) Comparative evaluation of phonetic distance metrics in a cross-lingual similarity task.

The rest of the paper is structured as follows. In Sect. 2 we review related work in the field of automated historical and multi-lingual toponym matching. In Sect. 3 we introduce our benchmark dataset and the synthetic dataset mined from wikidata. In Sect. 4 we detail the matching methods using transliteration and grapheme to phoneme conversion. Section 5 presents the experiments we performed and the results obtained and discusses them. We conclude in Sect. 6.

## 2 Related work

Several works have examined the efficacy of different string-based distance metrics for toponym matching. The most relevant of these is the work by Recchia and Louwerse (2013), which performed a comprehensive comparison between 21 different metrics on datasets extracted from the Geographic Names Server (National Geospatial-Intelligence Agency: Geographic Names Serve, 2024). The datasets contain romanized toponyms from 11 countries, and the toponyms were compared with their variants used in the same country. To the best of our knowledge, our work is the first to present a cross-language benchmark of *Semitic* languages and the first attempt to compare phoneme-based to transcription-based methods using direct comparison rather than through romanization.

Joshi et al. (2008) utilize transliteration methods to power a cross-lingual toponym search engine. However, they do not attempt to match the toponyms to each other as we do here, nor do they handle historical data. Transliteration was also used in previous work to transfer knowledge of named entities to low-resource settings. Klementiev and Roth (2006) propose a weakly supervised automatic discovery method that relies on a bilingual corpus between a source and target language pair and orthographic transliteration using a linear model. Their model selects candidate transliterations into the target language which then get re-ranked based on the best temporal alignment with the

source language entries. Bharadwaj et al. (2016), on the other hand, carry out phonemic transliteration using an attentional recurrent neural network. Their approach finds success in both the recognition of the monolingual named entities for Spanish and Turkish and in the transfer between Uzbek, Turkish, and Uyghur.

Aligning toponyms across large geographical databases, a task also known as *entity alignment*, has been attempted using various methods (see review Sun et al. 2019). However, none of these methods have used a matched cross-language benchmark to allow direct matching rather than through the romanized labels. Furthermore, to our knowledge, this work is the first to present such a benchmark for historical sources in the Middle East rather than modern place names. Geographical entity alignment algorithms use various types of methods, including textual, semantic, structural, spatial, and recently AI-based (e.g., Santos et al. 2018a). In this work, we focus on textual methods, often employed as part of larger solutions (Santos et al., 2018b). Specifically, we suggest two methods for direct comparison between related languages, showing them superior to methods relying on Romanization in this setting.

Several entity alignment benchmarks have been published in the past, mostly in the business (e.g., Zhang et al. 2020) and web (e.g., Zhang et al. 2022) domains. However, toponym alignment benchmarks are few and far between. General cross-lingual entity alignment (e.g., Sun et al. 2017) benefits from the fact that most entities, such as products, organizations and events, have descriptive labels and other properties (e.g. price, weight) that can be translated or compared. Toponyms, especially ones extracted from historical sources, have sparse information, often only the name of the place, the general geographical area, and some type information (river/city). Thus, toponym matching relies heavily on transliteration and orthographic comparison (Martins, 2011).

There are a few examples of same-language historical toponym-matching datasets. For example, Hastings (2008) align place names in English from recent USA history. The few examples we are aware of for cross-lingual historical toponym matching efforts are for places extracted from Dutch-German shipping maps (Ardanuy & Sporleder, 2017) and the comprehensive World Historical Gazetteer (WHG) (Grossner & Mostern, 2021), which collects toponym data from many different scholars. With respect to entity alignment, WHG allows the user to perform exact match orthographic comparisons between a dataset's toponym labels and external sources such as Wikidata and GeoNames. However, datasets in the WHG are not matched with each other, and thus do not enable scholars to enrich each other with information from sources of other languages and cultural backgrounds. Here, we present a benchmark enabling cross-cultural toponym matching and evaluation of direct cross-lingual methods.

### 3 A cross-lingual semitic toponym matching benchmark

In this section, we present our benchmark, comprising multiple datasets matched against each other in pairs. We shortly describe each dataset and comment on the prospect of finding matches between them.

We utilized four distinct datasets, two in Arabic and two in Hebrew. The initial Arabic dataset is a structured adaptation of *Kitāb Muʿjam al-Buldān* (The Countries Dictionary Book) Yāqūt (1977). This lexical masterpiece, authored in the early 13th century by the Muslim geographer Yāqūt al-Hamawī, provides a comprehensive account of various locales, predominantly within the Muslim world. The content encompasses diverse elements such as types of places, administrative hierarchies, geographical representations, and historical events. In a previous study, we employed a rule-based extraction approach to construct a systematically organized database containing many descriptive entities related to these places. Given the extensive nature of the lexicon, we opted to categorize the places based on geographical regions for more manageable analysis and presentation. The second Arabic dataset is sourced from the al-Ṭurayyā Project (Maxim Romanov: al-Ṭurayyā Project, 2024). Extracted from the “Atlas Du Monde Arabo-Islamique a l’Epoque Classique: IXe-Xe siècles” (The Arab-Islamic World and Classic Europe Atlas: 9th-10th Centuries) by Cornu (1983), this database comprises over 2000 toponyms. The dataset includes place names presented in both Arabic and Latin letters, accompanied by information on place types and, at times, a textual description of the location. The authors of the dataset employed a distinctive transliteration system characterized by a “one-to-one letter representation, with every Arabic letter transcribed distinctively” (Maxim Romanov: al-Ṭurayyā Project, 2024). In keeping with our methodology applied to Yaquṭ’s gazetteer, we parsed the al-Ṭurayyā dataset based on regional categorizations that align with our existing datasets. The first of the two Hebrew datasets is a list of Hebrew place names from the Medieval book of Travels of Benjamin of Tudela, composed in the 1170s. The text was digitized and annotated, and the places were geo-referenced for the building of the TraveLab tri-lingual digital edition TraveLab project (TraveLab, 2024). As the main Hebrew text, the 1840 Hebrew print edition by Benjamin et al. (n.d) was used, but variants were added from the aligned editions of both Arabic and English Translations and other Hebrew manuscript and print witnesses, which present a dynamically changing tradition of place naming. Though small (306 place entities), this dataset is closer in time to the Medieval modern sources, and though it does entail places from a part of Benjamin’s route in areas outside the Muslim world, there is still much overlap with the rest of his route. The second, much larger Hebrew collection is Kima project,<sup>1</sup> a gazetteer for historical and contemporary place names in languages written in the Hebrew script. The gazetteer provides a stable and shared reference for linking place names in digitized resources and documents and, where possible, attestation for the uses in the toponyms. Being mainly based on Hebrew print, its sources are predominantly early modern and modern. From Kima, we selected the toponyms from modern Spain, the countries of North Africa, and the Middle East that would be relevant to match the Arabic datasets.

Table 2 summarizes the pairs of datasets compared in this benchmark. For each dataset, the number of places is mentioned in parentheses. for example, the Tudela dataset contains 306 places. Some of the datasets are regional subsets of the original dataset. For example, when comparing Yāqūt and Kima in dataset pair number two,

<sup>1</sup> <https://data.geo-kima.org/>

we do this within the region of Al-Sham (The Levant, greater Syria), where these datasets contain 687 and 1899 places, respectively. We identified 30 exactly matched places out of 1,304,613 possible matches. We also compared Yāqūt and Kima datasets within the regions of Andalusia and Maghreb, resulting in 28 matched locations. We compiled the exact matches by running our tool using various combinations of its matching algorithms (see Sect. 4) and different thresholds and manually reviewing the identified matches. Verification was done by scholars familiar with the respective datasets. However, we do acknowledge that some of the possible matches may have been overlooked as it is not feasible to manually review over a million possible combinations of places. We invite future contributors to this benchmark to submit matched pairs that we may have overlooked.

## 4 Toponym matching methods

In this section, we detail our toponym matching methods, developed for direct comparison between closely related languages. We first detail our transliteration-based method (Sect. 4.1 followed by our phoneme-based method (Sect. 4.2).

### 4.1 Transliteration

It is common practice, in toponym matching, to transliterate or transcribe both toponym labels to Latin/English letters. The resulting labels are often referred to as *Romanized* labels (e.g., Recchia and Louwerse 2013). Some of these transliteration/transcription methods may supplant standard Latin script with diacritics to signify letters and sounds that do not exist in English. For example, many methods (e.g., Gibb 1998, Wehr 1979) use an accented *ī* to signify the Arabic letter *ي* (Ya). This may cause issues for edit-distance metrics such as Levenshtein et al. (1966) which consider this letter to be completely different from the regular *i*, which is used to transliterate the comparable Hebrew letter *י* (Yud). Therefore, we created a direct transliteration library that transcribes Arabic and Hebrew scripts using a rule-based linguistic-aware algorithm. The library is available as open-source code (MEHDIE Project: The translit-me source code repository, 2024) and as a python package (MEHDIE Project: The MEHDIE Transliteration Service and Python Package, 2024). The library employs simple one-to-one translation tables where no extra rules

**Table 2** The five dataset pairs used in the provided benchmark and the number of matches found between each pair

| Pairing | Dataset 1 (size)             | Dataset 2 (size)            | #Matches |
|---------|------------------------------|-----------------------------|----------|
| 1       | Yaqut - Andalus/Magreb (484) | Kima - Andalus/Magreb (559) | 28       |
| 2       | Yaqut Al-Sham (687)          | Kima - Al-Sham (1899)       | 30       |
| 3       | Kima - Al-Sham (1899)        | Thuraya - Al-Sham (291)     | 21       |
| 4       | Tudela (306)                 | Althurayya (2241)           | 18       |
| 5       | Damast (447)                 | Tudela (306)                | 32       |

are needed and more elaborate rule-based transcriptions where the transliteration differs according to letter position and other contexts. For example, the Hebrew letter ה (Heh) would normally be transliterated to *h* (Ta), but if it is at the end of the word, to *h* (Ta Marbuta). The library also generates multiple variants when the transcription has unsolvable ambiguity. For example, a ה (daleth) letter in Hebrew may be transliterated into both ד (dāl) and ד (dād). The user may subsequently choose which variant to use. We arbitrarily chose the first generated variant in this work for consistent experimental results. In the experiments performed, we employ the Jaro (1989) and Levenshtein et al. (1966) distance metrics, defined as follows.

**Definition 1 (Jaro Distance Metric)** Given two strings  $s_1$  and  $s_2$ , the Jaro similarity score,  $J(s_1, s_2)$ , is calculated using the following formula.

$$J(s_1, s_2) = \frac{1}{3} \left( \frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m - t}{m} \right)$$

Where  $m$  is the number of matching characters (two characters from  $s_1$  and  $s_2$  are considered matching if they are the same and not farther apart than  $\max(\text{len}(s_1), \text{len}(s_2))/2 - 1$ ),  $t$  is half the number of transpositions (a transposition occurs when the same character appears in both strings but in different orders),  $|s_1|$  and  $|s_2|$  are the lengths of the strings  $s_1$  and  $s_2$ , respectively. The Jaro distance is then defined as  $1 - J(s_1, s_2)$ .

**Definition 2 (Levenshtein Distance Metric)** The Levenshtein distance between two strings,  $a$  and  $b$ , is defined as the minimum number of single-character edits (insertions, deletions, or substitutions) required to change one string into the other. Formally, the distance  $L(a, b)$  can be defined recursively as follows:

$$L(a, b) = \begin{cases} |a| & \text{if } |b| = 0, \\ |b| & \text{if } |a| = 0, \\ L(\text{tail}(a), \text{tail}(b)) & \text{if } \text{head}(a) = \text{head}(b), \\ 1 + \min \begin{cases} L(\text{tail}(a), b), \\ L(a, \text{tail}(b)), \\ L(\text{tail}(a), \text{tail}(b)) \end{cases} & \text{otherwise.} \end{cases}$$

Here,  $|a|$  and  $|b|$  denote the lengths of strings  $a$  and  $b$ , respectively;  $\text{tail}(S)$  denotes the string  $S$  without its first character, and  $\text{head}(S)$  denotes the first character of string  $S$ .

## 4.2 Grapheme-to-Phoneme conversion

We employ Grapheme-to-Phoneme (G2P) conversion and subsequently apply phonetic distance measures as an alternative approach to cross-lingual toponym matching. This allows to take into consideration the fact that phonemes, i.e., speech sounds (e.g., consonants and vowels) do not differ from each other to the same



extent. Contemporary phonological theory characterizes phonemes based on phonological features, including phonation, airstream mechanism, and manner and place of articulation (Catford & Esling, 2006). When language change distances the phonology of otherwise related languages like Arabic and Hebrew from each other, it is more common for only a subset of phonological features to diverge. G2P and phonetic distance measures allow a more accurate and fine-grained comparison between toponyms than simple transliteration.

We use the `g2ps` library,<sup>2</sup> part of the `LanguageNet` project,<sup>3</sup> as our G2P conversion tool. This is due to its coverage in our target languages of English, Hebrew and Levantine Arabic compared to other G2P tools, which are not compatible with all these languages or do not provide truly phonemic transcription.<sup>4</sup> Training lexicons provided by `g2ps` can be used to train `Phonetisaurus`,<sup>5</sup> a weighted finite-state transducer on mappings between graphemes and corresponding phonemes (Novak et al., 2016). We augment this transducer with a series of sensible post-processing rules defined to enable more accurate transcriptions. We examine the contribution of these rules to the matching accuracy in Sect. 5.

We compare the resulting phonemes using the distance module of the `PanPhon` library (Mortensen et al., 2016).<sup>6</sup> The library converts input phonemes (represented using IPA symbols) into two possible representations: Dolgopolsky primes (Dolgopolsky, 1986) and phonological feature vectors. It also contains various distance measures based on edit distance, i.e., the number of edit operations required to transform one string to another. Dolgopolsky primes are groupings of individual phonemes into distinct sound classes with categories representing labial obstruents (e.g. /p/ and /b/), laryngeals (e.g. /h/ and /ɦ/), and vowels (e.g. /e/ and /ə/). Once the phonemes are assigned to their corresponding prime, the resulting sequences can be compared using Levenshtein distance. The drawback of Dolgopolsky primes is that they assign phonemes into mutually exclusive categories, thus not directly comparing phonological features. There is, for instance, a separate category in Dolgopolsky primes for coronal fricatives, coronal affricates, and other coronal obstruents, sounds that are obviously similar in their manner of articulation.

On the other hand, phonological feature vectors represent phonemes directly using abstract phonological features linguists break them down to. These may refer to articulatory features, i.e., how a sound is formed, including categories such as *lateral*, *nasal*, *voiced*, or *labial* among all. Additional phonological features express the *consonantal* nature of a sound, or its *syllabicity*.<sup>7</sup> Features values are encoded as either positive, negative, or unspecified. All IPA symbols can be broken down into these feature vectors, and the library contains various distance measures that can operate on these vectors, including *feature edit distance*, *Hamming feature edit distance*, and *weighted feature edit distance*. These distance measures differ in the

<sup>2</sup> <https://github.com/uiuc-sst/g2ps>

<sup>3</sup> <http://www.isle.illinois.edu/sst/research/darpa2015/index.html>

<sup>4</sup> See <https://github.com/topics/grapheme-to-phoneme>

<sup>5</sup> <https://github.com/AdolfVonKleist/Phonetisaurus>

<sup>6</sup> <https://github.com/dmort27/panphon>

<sup>7</sup> See `PanPhon` phonological features <https://github.com/dmort27/panphon/tree/master/panphon/data>

associated cost of feature edits. The first two assign fixed costs to changing any features, while weighted feature edit distance considers that certain sound changes are more common cross-linguistically than others. For instance, voicing and devoicing a consonant from /p/ to /b/ or /g/ to /k/ is extremely common, as is the lengthening and shortening of vowels. The low cost of changing these phonological feature values reflects this. On the other hand, making a continuant, like an /s/ sound, into a non-continuant, such as /t/ or vice versa, has a significantly higher cost, modeling how unlikely this sound change is.

Toponym matching across related languages, such as Hebrew and Arabic, might benefit greatly from considering how close the individual sounds to each other are phonologically. When using an appropriate similarity threshold, we hypothesize that phonologically-based toponym matching might improve on merely string-based approaches.

## 5 Experiments

In the following Section, we perform two experiments. In the first experiment (Sect. 5.1). We empirically evaluate the performance of our transliteration and phonetic methods over our benchmark. We perform toponym matching and report on the performance of our transliteration-based and phonetic-based methods on this benchmark.

In a second experiment (Sect. 5.2), we perform a more detailed phonetic and orthographic distance analysis. We do so by comparing the transliteration-based and phonetic-based metrics with state-of-the-art string-based metrics over a large and diverse dataset, namely - Wikidata toponyms in modern variants of Arabic and Hebrew. We perform the comparison using different distance metrics.

### 5.1 Toponym matching over historical dataset pairs

#### 5.1.1 Setup and metrics

Toponym matching experiments were performed using our prototypical MEHDIE (MEHDIE: The MEHDIE toponym matching tool, 2024) toponym matching tool. Normally, MEHDIE employs both the transliteration-based and the phonetic methods. However, we used the configuration options to run experiments with only one of the methods at a time. Configuration settings were varied to examine the sensitivity of the results to different threshold settings.

The MEHDIE tool allows matching a given dataset to another dataset by performing parallelized pair-wise comparisons between toponyms distributed over instances of the matching system, which are spun up by demand by Google Cloud Run infrastructure.

The tool first performs a preprocessing step on both datasets where toponyms (for example, in Hebrew) and their variants, as supplied with the dataset, are transliterated to the opposite language (for example, Arabic) and Romanized. The

tool then performs a blocking step to reduce the number of comparisons, which is common in large-scale matching problems (Papadakis et al., 2020). However, general-purpose blocking methods rely on the linguistic similarity of the matched entities, which is difficult to use in our case, where the information is limited to the entity's name, which is often linguistically opaque, and the entity's location. The tool, therefore, employs a simple clustering step where every name is assigned to groups according to the existence of one of a set of similar-sounding letters in the name. For example, all names from both datasets with at least one of the letters *p, b, f* would be grouped together and eventually matched. The assumption is that if the pair of toponyms do not share any related letter in the Romanized version, they are not relevant to each other. The blocking step reduces the number of comparisons, on average, by half.

Following the blocking step, the tool performs a pairwise comparison over transliterated variants of the toponym from the same script language (Hebrew/Arabic) using the Jaro metric. Results over the given thresholds continue to perform several pre-processing steps next step, which is a phonetic comparison using the Hamming feature distance metric over the IPA representation of the toponym. Finally, if location information is available for both matched toponyms, we filter out those pairs whose distance exceeds the given threshold. Phonetic threshold values are varied between  $[0.85, 0.9, 0.95]$ , orthographic thresholds between  $[0.7, 0.8, 0.9]$ , and distance thresholds between  $[10, 50, 200]$ .

Results are provided in terms of *Precision*, *Recall*, *F-1*, and *F-5* defined as follows. Precision is the proportion of true positives of the number of matched pairs the tool returns. Recall is the proportion of true positives of the number of matched pairs known to be in this dataset pair. F-1 is the harmonic mean of these two measures, penalizing imbalanced results (e.g., low recall and high precision) more than the standard mean. F-5 is a recall-favoring version of the F-measure. The equations of the general F-measure, F-1 and F-5, are given below with P and R designating *Precision* and *Recall*, respectively.

- **F- $\beta$  Score** provides a means to balance precision and recall in a single metric, using a weighted harmonic mean. It is expressed as:

$$F_{\beta} = (1 + \beta^2) \cdot \frac{P \cdot R}{(\beta^2 \cdot P) + R} \quad (1)$$

The  $\beta$  parameter determines the weight of recall in this balance, with values greater than 1 favoring recall, thus prioritizing the identification of all actual matches.

- **F-1 Score** is the harmonic mean of precision and recall with equal importance, suitable for scenarios where both metrics are equally valued:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} \quad (2)$$

- **F-5 Score** emphasizes recall five times more than precision by setting  $\beta = 5$ . This is particularly useful in our case, where the discovery of new matches is

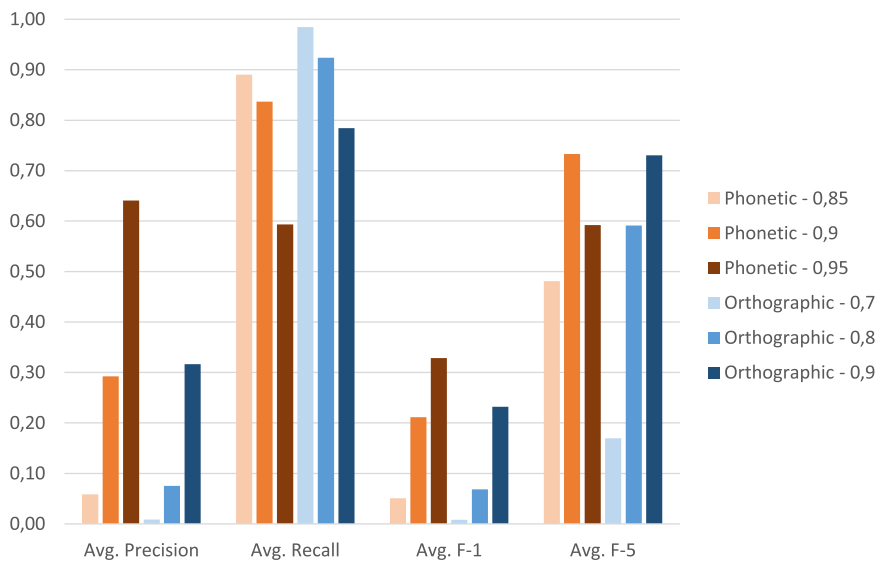
highly sought after by our users, and they are willing to pay for it by reviewing more (mostly false) results.

$$F_5 = (1 + 5^2) \cdot \frac{P \cdot R}{(5^2 \cdot P) + R} \quad (3)$$

### 5.1.2 Results

The complete set of results is provided as an electronic supplement to this paper. In the following, we present some of the main findings. Figure 2 presents the different metric values for different threshold settings applied to the two methods. As expected, an increasing threshold improves precision at the expense of recall. The phonetic method seems to be severely impacted by lowering the threshold. A reduction of 0.1 (from 0.95 to 0.85) in the phonetic threshold improves recall by 0.3 and reduces precision by more than half. The transliteration-based method, by comparison, gains a 0.06–0.14 in recall and suffers only a 0.07–0.024 reduction in precision from the same reduction. Regarding F-1, the low precision values coupled with high recall rates cause the F-1 measure to be relatively low in both methods at 0.1–0.32 and to be maximized in high-threshold scenarios where the precision is maximized. Optimal F-5 values are obtained for a threshold of 0.9 in both the orthographic and the phonetic methods.

Figure 3 compares the impact of different distance thresholds on the different dataset pairs. The figure presents results in terms of the maximal F-5 result obtained



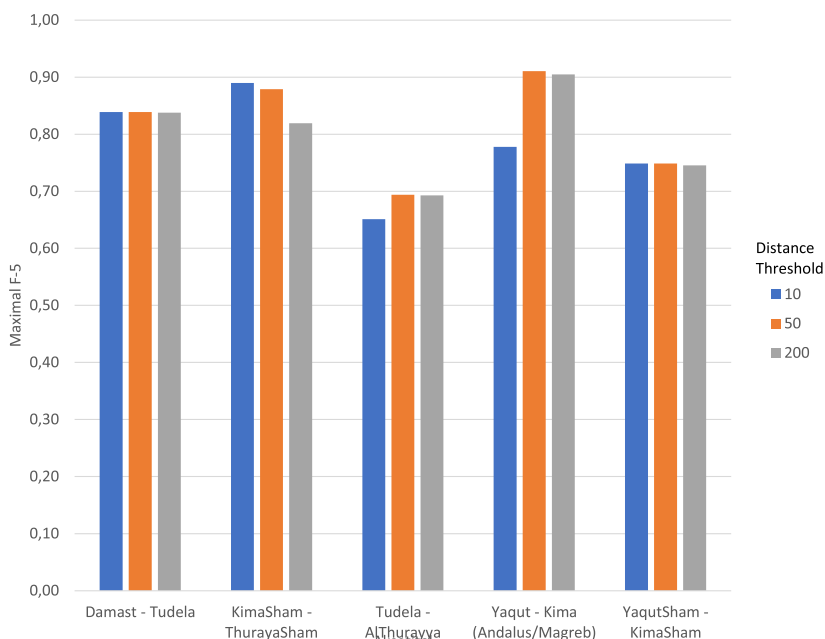
**Fig. 2** Comparison of different threshold values for both methods

across all threshold settings and comparison methods. Results indicate that applying a more restrictive distance threshold generally reduces performance. This emanates from the reduced recall, which our users value. This general trend does not hold in the case of Kima-Thurayya, where applying a restrictive threshold improves the F-5 score somewhat, owing to the very low precision caused by the more lenient distance thresholds.

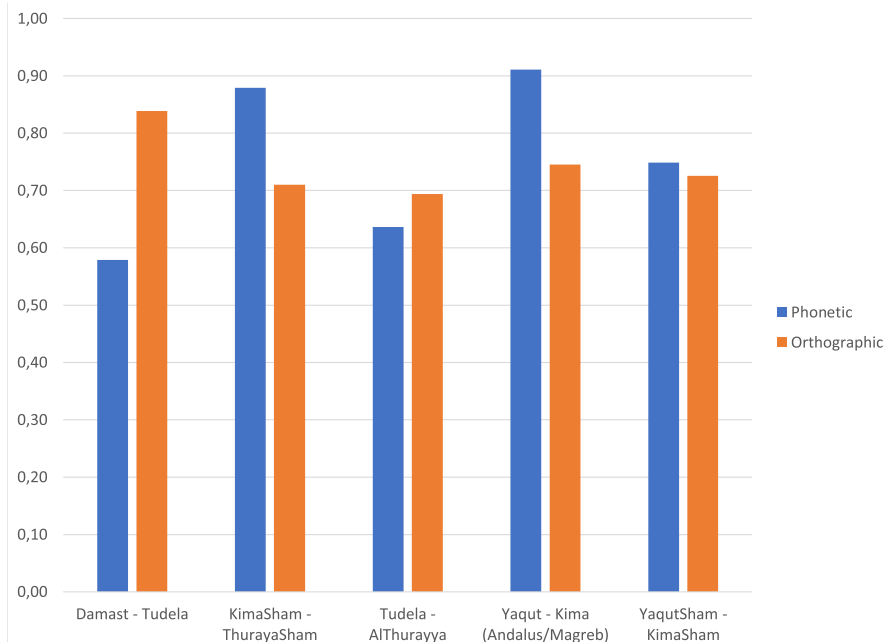
In Fig. 4, we compare the two matching methods across the five dataset pairs in F-5 terms, reporting the maximal result overall threshold and distance threshold settings. Results are mixed, with dataset pairs involving the Tudela dataset showing better performance for the orthographic method, while in the other dataset pairs, the phonetic method is superior.

### 5.1.3 Discussion

As F-5 better represents users' preferences in our scenario, it is interesting to see that the best F-5 is obtained at a threshold of 0.9 for both the orthographic and phonetic methods. Combined with the inconclusive comparison results over different dataset pairs, this result indicates that users should opt to use both methods in combination to maximize the number of matches found. The fact that restrictive distance thresholds harm performance can partially be attributed to our choice of the F-5 metric owing to our users' preference for high recall and relative tolerance of low precision. However, it can also be traced to the fact that location data for historical sources is inaccurate. In some cases, the historical place has a location that is



**Fig. 3** Best result in terms of F-5 by matched dataset pair and distance threshold



**Fig. 4** Best result in terms of F-5 by matched dataset pair and method

accurately mapped to research-based evidence of settlement. Still, in others, it may be approximated using reported travel times from other locations or using inaccurate historical maps (Tucci & Giordano, 2011). Although it is common for place names in the ancient Middle East to be repeated (e.g., الطيرة in Israel,<sup>8</sup> the Palestinian Authority,<sup>9</sup> and Syria<sup>10</sup>), it seems the benefits of using a more lenient distance threshold outweigh this risk. The phonetic similarity tool between Semitic languages proved more efficient in places like Tiberias in the Yāqūt and Kima in al-Sham that are phonetically closer in Hebrew (טבריה-Tveria) and Arabic (طبرية-Tabaria) than in an orthographic comparison of the Romanized versions. In another example Gaza (غزة-Ghaza in Arabic and ג'ז'א-Aza in Hebrew) were matched based on two criteria: phonetic title similarity by confidence 0.97222 and exact geographic similarity [34.45, 31.516]. The phonetic tool adeptly disregards grammatical nuances such as the Arabic definite article "ال", leading to enhanced statistical accuracy. Consequently, it yields high matches in locations where "ال" appears in Arabic but not in Hebrew, as exemplified by Zarqa (الزرقاء-זרקה) and Ramla (الرملة-רמלה). Nevertheless, the transliteration-based matching tool was also able to identify pairs that the phonetic tool missed. For example, the city of Zaragoza in Andalusia (Spain) is spelled by Yāqūt (سرقسطة-Saraqustah) and by Kima's gazetteer (סרגוסה-Saragosa). Here, the phonetic similarity is rather distant. However, since the orthographic

<sup>8</sup> <https://www.wikidata.org/wiki/Q167594>

<sup>9</sup> <https://www.wikidata.org/wiki/Q12189847>

<sup>10</sup> <https://www.wikidata.org/wiki/Q12189848>

comparison is much more computationally efficient, we can use it over multiple name variants supplied by the sources and take the maximal similarity score. In this case, the algorithm found in one of the variants that existed in Kima originally **סרקסטח** (Sarakastah) whose direct transliteration to Arabic orthographically matches that of Yāqūt.

In practice, since both the direct transliteration and the phonetic tool succeed in capturing some of the matches, and following our users' preference for recall, we use a union of the results from both approaches when returning results in the matching tool.

## 5.2 Comparing orthographic distances between modern toponym names in Hebrew and Arabic

### 5.2.1 Dataset description

We queried wikidata using the SPARQL query depicted in Listing 1 to construct our test case. The query uses the *wdt:p625* location property to identify concepts that may be toponyms. These are then filtered for those concepts that have a label in both Arabic and Hebrew. The query was run over Wikidata using the qlever query engine (Bast and Buchhold, 2017) and resulted in 41560 toponym label pairs.<sup>11</sup>

Listing 1 SPARQL Query to fetch Hebrew and Arabic labels

```
SELECT ?place ?placeLabel_he ?placeLabel_ar WHERE {
  ?place wdt:P625 ?location . # P625 is the property for coordinate
    location
  ?place rdfs:label ?placeLabel_he FILTER(LANG(?placeLabel_he) = "he
    ") .
  ?place rdfs:label ?placeLabel_ar FILTER(LANG(?placeLabel_ar) = "ar
    ") .
}
```

We then perform several pre-processing steps. The processing code, as well as both the raw and processed files, are available as an electronic appendix to this paper. We begin by removing all places for which either the Hebrew or the Arabic label comprises more than two words. This is done to filter out locations that are not real places but locations of events or businesses. Such as the *Academy of Social Sciences of the Central Committee of CPSU*<sup>12</sup> or *Air France Flight 296*.<sup>13</sup> Although this method may sound restrictive, one should remember that prepositions in Hebrew and determiners in both languages are prefixed to the word. For example, *the Jordan river* is in Hebrew (transliterated to Latin characters) *Nahar Hayarden* where *Ha* is the equivalent to *the* and in Arabic it is *Naher Alurdun* where *Al* is the

<sup>11</sup> <https://qllever.cs.uni-freiburg.de/wikidata>, retrieved November 9th 2023.

<sup>12</sup> <https://www.wikidata.org/wiki/Q4059245>

<sup>13</sup> <https://www.wikidata.org/wiki/Q406486>

| ID      | Hebrew        | HE IPA       | HE Translit AR | Arabic        | AR IPA           | AR Translit HE |
|---------|---------------|--------------|----------------|---------------|------------------|----------------|
| 1015647 | קלרסדורב      | klrksdɔrf    | كلركسدورف      | كلركسدورب     | kljrksdu:rb      | כלירכסדורב     |
| 1015654 | סנטו דומינגו  | snto domiŋv  | سنطو دومينجو   | سانتو دومينغو | sa:ntu: dwme:nɣw | סאנתו דומינע'ו |
| 1015672 | וואנגני       | vɔŋnvi       | ووانجنوي       | وانجانوي      | awa:ndʒa:nwj     | ואנג'אני       |
| 1015699 | לירה          | liɐa         | ليرة           | لیرا          | ljra:            | לירא           |
| 1015727 | מבאלה         | mɐʔla        | مباله          | מבאלי         | mɐba:lɐ          | מבאלי          |
| 1015773 | פורירואה      | foriroʔa     | فوريرواة       | בוריריוא      | bo:ri:re:awa:    | בוריריוא       |
| 1015796 | ויקטור הארבור | wiqtɔr hʔrɔr | ويكتور هاربور  | פיקטור הארבור | fi:ktu:r ha:rɔwr | פיקטור הארבור  |
| 1015805 | צ'אנגשו       | tsʔŋʃɔ       | ضانغشو         | תשאנגשו       | tʃa:nyʃaw        | תשאנע'שו       |
| 101583  | תבאי          | tbɔi         | تبائي          | טבייה         | tʔe:ba           | טיבה           |
| 1015916 | הדיבו         | hdivo        | هديبو          | חדייבו        | hdajbu:          | חדיבו          |

**Fig. 5** Result of preprocessed Wikipedia dataset

determiner. Thus, in both Hebrew and Arabic, this step will not remove many actual Toponyms but is very helpful to avoid skewing the results by measuring distances between long sentences describing events and businesses that are not common in our benchmark datasets that contain names of places. Furthermore, these sentences are better compared with translation as they contain common nouns and adjectives rather than proper nouns.

We remove language tags (e.g., '@ar') from the toponyms and then perform transliteration of each toponym to its opposite language using the specialized middle-east language transliteration package *translit-me*.<sup>14</sup> The original Arabic and Hebrew labels are then converted into an IPA phonetic representation. Figure 5 shows an excerpt of the result.

### 5.2.2 Metrics

As a baseline comparison, We use string comparison metrics for toponym matching from those compared by Recchia and Louwerse (2013). The authors compared 21 string comparison methods over datasets containing Romanized toponyms from 11 countries. We chose the top three best-performing metrics the authors identified for Arabic Toponyms (Saudi Arabia datasets) as they are the closest linguistic and geographic to our datasets. We compare the performance of *Skip-grams*, *Editex*, and *Syllable* over the Romanized versions of both Hebrew and Arabic labels. Using standard Jaro and Levenshtein distance metrics, we also compare strings in the same character set, i.e., Hebrew or Arabic, where one is the original label and the other is transliterated. The following are the definitions of these metrics based on Recchia and Louwerse (2013). We employ the methods as implemented by a python port<sup>15</sup> of the FEBRL package (Christen, 2008).

#### *Skip-grams*

<sup>14</sup> <https://pypi.org/project/translit-me/>, version 1.0.4, retrieved October 30th, 2023.

<sup>15</sup> <https://github.com/thomaswyrick/febrrl>, retrieved November 12th, 2023.



We use the method first proposed by Keskustalo et al. (2003) where common bigrams of skip size one and two are counted and divided by the total number of these bigrams in the shorter place name.

### ***Smith-Waterman***

This method (Smith & Waterman, 1981) employs a variation of Levenshtein distance where letters of the same sound group, e.g. (aeiou, bpv, dt, etc.) have a lower edit distance than if they belong to different letter groups. This method resembles our phonetic method but is more crude as it groups together many unrelated letters and is based on the English language pronunciation of these characters.

### ***Syllable***

The syllable alignment algorithm (Gong & Chan, 2006) analyzes syllables instead of individual characters. It transforms characters in place names into groups, forming sequences that represent syllables. The algorithm then calculates the least costly way to change one sequence into another, using a set of weighted operations on both character groups and syllables.

### ***Phonological distance measures***

In order to carry out toponym matching based on phonological similarity, we convert candidate toponyms to phonological vectors, where each element is a phonological feature of the sound (e.g., labial). We then use various distance measures in the PanPhon library (Mortensen et al., 2016) to compare them as described in Sect. 4.2. All of these distance measures are based on the concept of edit distance, i.e., the number of substitutions required to transform one toponym into another. In the following, we describe the fundamental categories.

For the *feature edit distance*, every substitution has an associated *cost* of 1/24: i.e., normalized by the total number of phonetic features. The same cost is associated with any insertion or deletion. In cases where the edit goes from an unspecified to a specified phonological feature, the edit cost is half of the above. This is necessary since certain features only make sense in the context of other features. Obstruents like /t/, for instance, cannot be specified for the feature strident, but once they change into a fricative such as /θ/, they have to become specified. Since this involves additional feature changes anyway—from −continuant to +continuant—, this prevents inflating edit costs.

$$d_{\text{feature}}(\text{ins, del, sub}) = \frac{1}{\|\mathbf{f}\|} \left( \sum_i \sum_j \frac{1}{2} \|\mathbf{s}_i - \mathbf{t}_j\| \right) \quad (4)$$

The cost of all edit operations between two phonemes is captured in Eq. 4. Vectors  $\mathbf{s}_i$  and  $\mathbf{t}_j$  stand for the  $i$ -th and  $j$ -th members of the phonological vectors  $\mathbf{s}$  source phoneme and  $\mathbf{t}$  target phoneme. The vector  $\mathbf{f}$  represents all possible feature values, and its magnitude is 24. Since 1, −1, and 0 represent positive, negative, and unspecified values for each feature, the normalizer term  $\frac{1}{2}$  ensures that the cost of changing from specified to unspecified and vice versa is less than changes between specified values. Again, this prevents the inflation of edit costs when certain feature changes are necessarily accompanied by other feature changes.

In the case of *Hamming feature edit distance*, insertions and deletions cost 1 without normalization, while feature edits and edit from unspecified to specified all cost 1/24 (see Eq. 5).

$$\text{Difference} = \begin{cases} 1 & \text{if } s_i \neq t_i \\ 0 & \text{if } s_i = t_i \end{cases} \quad (5)$$

$$d_{\text{hamming}}(\text{sub}) = \frac{1}{||\mathbf{f}||} \left( \sum_i \text{Difference}(s_i, t_i) \right)$$

Finally, *weighted feature edit distance* takes into account the probability of the kinds of feature edits determined by how likely these are cross-linguistically, but does not carry out normalization. Cross-linguistically frequent feature changes, such as changes in voicing or vowel length, have an associated cost of 0.125, less common changes come with a cost of 0.25 or 0.5, whereas rarer ones such as change in syllabicity have a cost of 1. See Table 3 for a comparison between the categories.

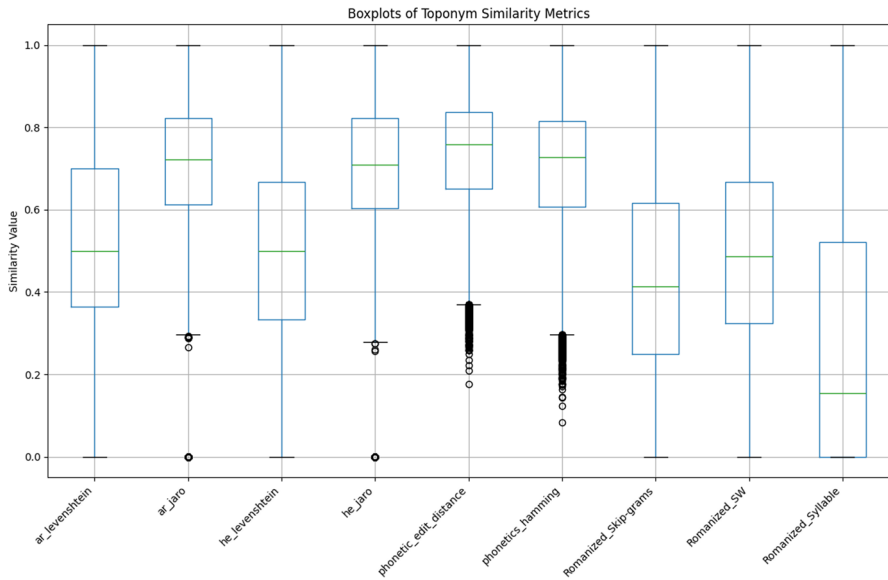
### 5.2.3 Results

Figures 6 and 7 present the box plots and density plots of the different metrics over the wikidata datasets. *ar*-prefixed metrics are measured between the original Arabic place name and the Hebrew place name that was transliterated into Arabic. Similarly, *he*-prefixed metrics compare between the original Hebrew toponym and the one transliterated into Hebrew. *Romanized*-prefixed metrics compare between the romanized versions of both labels (i.e., romanized Hebrew and romanized Arabic). Metrics prefixed with *phonetic* are calculated over the IPA representations of the toponyms. In general, a result closer to one is more desirable. Thus the perfect distribution in Fig. 6 would be a dot on the 1.0 horizontal line, and in Fig. 7, it would be a vertical line at 1.0. Table 4 summarizes the information shown in the box plots in numerical form. In bold, we highlight the highest value for the quartile, mean, and minimum values which is consistently that of the phonetic feature edit distance metric. A close second and third best are the two Jaro metrics directly comparing Hebrew and Arabic labels with their transliterated counterparts with very similar performance.

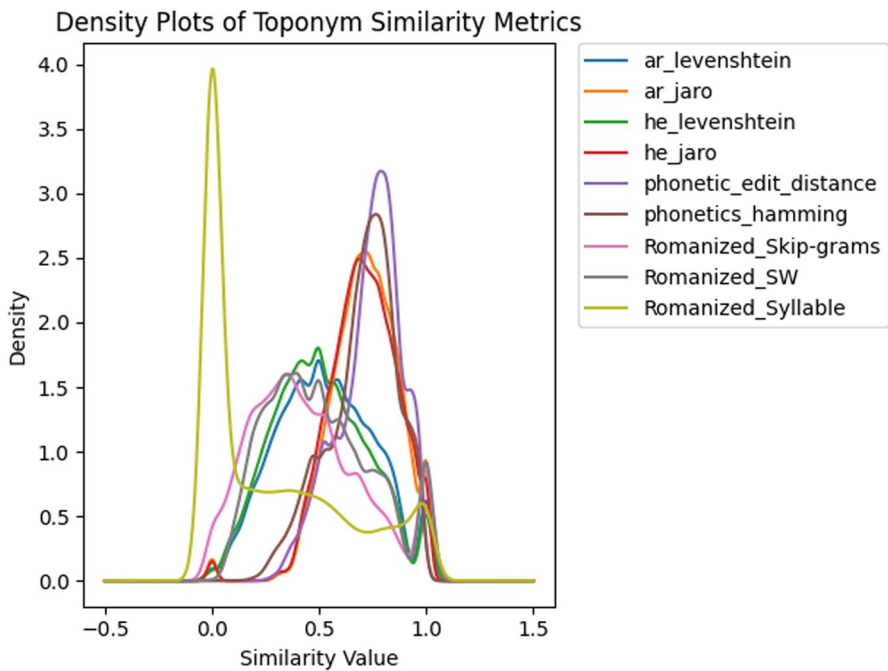
**Table 3** Comparison of the costs associated with the edit distances defined in the PanPhon library

| Edit distance    | Substitution (specified) | Substitution (unspecified) | Insertion/deletion  |
|------------------|--------------------------|----------------------------|---------------------|
| Feature          | 1/24                     | 1/48                       | 1/24                |
| Hamming feature  | 1/24                     | 1/24                       | 1                   |
| Weighted feature | 1/8, 1/4, 1/2 or 1*      | 1/8, 1/4, 1/2 or 1*        | 1/8, 1/4, 1/2 or 1* |

The asterisk marks that the exact edit cost depends on the associated features



**Fig. 6** Box plots of string similarity and phonetic similarity Metrics over Wikidata toponyms



**Fig. 7** Density plots of string similarity and phonetic similarity Metrics over Wikidata toponyms

**Table 4** Descriptive statistics of similarity metrics over the wikidata dataset

| Metric | AR Ln | HE Ln | AR Jaro | HE Jaro | Skip-G | SW   | Syllable | $\phi$ ED   | $\phi$ Hg |
|--------|-------|-------|---------|---------|--------|------|----------|-------------|-----------|
| Mean   | 0.53  | 0.51  | 0.71    | 0.71    | 0.45   | 0.50 | 0.29     | <b>0.74</b> | 0.70      |
| Std    | 0.23  | 0.22  | 0.16    | 0.16    | 0.26   | 0.24 | 0.33     | 0.15        | 0.16      |
| Min    | 0.00  | 0.00  | 0.00    | 0.00    | 0.00   | 0.00 | 0.00     | <b>0.18</b> | 0.08      |
| 25%    | 0.36  | 0.33  | 0.61    | 0.60    | 0.25   | 0.33 | 0.00     | <b>0.65</b> | 0.61      |
| 50%    | 0.50  | 0.50  | 0.72    | 0.71    | 0.41   | 0.49 | 0.15     | <b>0.76</b> | 0.73      |
| 75%    | 0.70  | 0.67  | 0.82    | 0.82    | 0.62   | 0.67 | 0.52     | <b>0.84</b> | 0.81      |
| Max    | 1.00  | 1.00  | 1.00    | 1.00    | 1.00   | 1.00 | 1.00     | 1.00        | 1.00      |

*Ln* Levenstein Distance, *Skip-G* skip Grams, *SW* Smith-Waterman,  $\phi$  Phonetic, *ED* Edit Distance, *Hg* Hamming Distance

## 5.2.4 Discussion

Both the direct transliteration (Hebrew  $\leftrightarrow$  Arabic) with Jaro and the IPA phonetic algorithms perform considerably better than the distance metrics applied over the Romanized toponyms. Jaro is somewhat hindered by the limitations of the Jaro algorithm when handling very short strings, and the limitations of the dataset, which contains many non-Middle-eastern names that require various apostrophes and forced letter pairs. For example, the Chinese city of Changshu<sup>16</sup> (Fig. 5 third to last row) whose first sound is transliterated by the editors of wikidata using the available letters in Arabic and Hebrew to a T+Sh construct. These letter combinations cause the match distance to grow artificially in replacement-based algorithms such as Jaro. Interestingly, the Levenstein distance metric performs poorly in both languages.

We inspected the cases with the largest gaps between the phonetic and the Jaro distance metric. All of the results could be explained by one or more of the following reasons. Several cases reveal actual errors in Wikidata (as, for example, the case of "Bethlehem, Georgia"(Q3261160) which was erroneously named 'Georgia' in Hebrew (we corrected it on Dec.19, 2023) or "Jaffa port"(Q2911475) which erroneously contained the Hebrew word for Israel in the field of the Arabic variant (corrected Dec.29 2023). The distance criterion even tracked cases of Wikidata vandalism, as in the case of the town Yeruham (Q1687708), that in early November 2023 was renamed (Palestine), a renaming that was reverted since.

In many other cases, the names given in the different languages are in fact different names for the same place. Two examples are revealing: the island of Imbros (wikidata entity Q658437) bears the same name (إمبروس) in Arabic as in English and other Wikipedia languages, which chose to follow the older Greek name Ἰμβρος. The Hebrew name גִּזְרֵי אֶרֶד, however, follows the official current Turkish name of the island Gökçeada since 1970. Another example is Wikidata entity Q581478: The Arabic name is derived from the Czech name, Břeclav, which in turn is named after the founder of the local castle, Duke Bretislav I. The Hebrew form preserves the former German name which was probably derived from the name of a Slavic tribe

<sup>16</sup> <https://www.wikidata.org/wiki/Q1015805>

which lived in the area. The example of Q826132 shows another known phenomenon in toponym history, in which a hyponym (a name that refers to a part of a certain region) like (Lagat/Laja, اللجة) became synonymous with its hypernym (the name of the entire region), as in this case Trachon / (טרַכּוֹן). The Arabic and Hebrew choices of the name each corresponds with one of the options. Behind most of these cases lies the cultural and political history of place naming, which is occasionally reflected in the choice of Wikidata editors.

Another common reason for large reported differences between the Arabic and Hebrew names occurs when the root name is basically the same, but in one of the languages the name includes a type word (such as Wadi, Bridge, Mountain, Mound, River, cave) and not in the other. Thus, while the 'Fengtai District' of Beijing (Q393831) is named simply *فَنغْتَاي* in Arabic, the Hebrew name also includes the Hebrew word for 'district': *רִבְעָה פִּנְגְּטַאִי*. The river Soča, on the other hand, is simply named *סוּצ'ה* in Hebrew while the Arabic includes the term for 'river', *نهر سوكا*.

While the choice to include the type word in the name seems random and occurs in both languages, a similar reason for large distance between the variants stems from linguistic reasons: often, the Arabic name of a place will include the determiner *ال* as an inherent part of the toponym. Thus the Iraqi city of Najaf (Q168193) is named simply *نَجَف* in Hebrew but *النجف* - "the Najaf" in Arabic, and the determiner letters prefixed to the Arabic version cause the distance between matching letters to be too great for the Jaro algorithm to take into account. Here, we encounter a peculiar property of the Jaro metric where the allowable distance between matching letters is proportional to the length of the shortest comparison term. This may emanate from the fact that Jaro was developed for comparing paragraphs of text rather than single words. Thus, for a three-letter word like the Hebrew version of this toponym, the two-letter determiner prefix distances the matching Arabic letters too far away to be included in the Jaro calculation, resulting in a Jaro score of zero. For the phonetic versions of these two toponyms, this problem does not exist since we have implemented, as one of the post-processing rules, a rule that removes the determiner from the phoneme. Applying a similar rule for type names (river, county, etc.) could improve both metrics' performance and be justified in a toponym-focused matching system such as ours.

A third type of difference between the names in the two languages stems from the inclusion of *Matres Lectionis*. While both the Arabic and Hebrew scripts lack vowel letters in principle, they may occasionally use specific letters to indicate vowels. In Modern Hebrew, the adoption of full spelling means that the Hebrew name is more likely to include more letters, as the case of Kutum (Q3317346), for which the Hebrew name includes the five letters parallel to the English name, and the Arabic name only the three consonants k,t,m. While for string similarity metrics the distance of two letters is substantial, the phonetic distance metrics accords smaller weights to differences in vowel letters, and thus is more likely to match the two names. Here, it is important to note that we experimented with removing these *Matres Lectionis*, and the transliteration package (MEHDIE Project: The MEHDIE Transliteration Service and Python Package, 2024) provides such an option. However, we found that the reduction in toponym length makes

every variation significantly impact the similarity score, offsetting any gains from the reduced uncertainty.

The most frequent explanation for the differences between the phonetic and Jaro similarity metrics also exhibits the strength of the phonetic approach: the case of Pest (Q210205), the eastern part of Budapest, is a clear example: in both Hebrew and Arabic the name is written with three letters for the consonants, but though the names are in principle the same, none of the letters can, however, be unequivocally transliterated to its parallel letter in the other language. The phonetic approach is sensitive to the closeness between the bilabial plosives p (פ) and b (ב), between the sibilant second letters and the Voiceless alveolar plosive third letters.

## 6 Conclusion

In this work, we have presented a first-of-its-kind benchmark for cross-language toponym matching between Hebrew and Arabic historical toponyms. The benchmark comprises five dataset pairs from four different gazetteers of the Middle East, Magreb, and Andalus areas. We have further presented and evaluated two novel approaches to this problem. Our transliteration-based approach and our phonetic-based approach were explained and evaluated, both on our benchmark task and on a large-scale synthetic task created using Wikidata, and compared to traditional methods relying on the romanization of toponyms and the use of edit-distance and n-gram-based string comparison methods. Results indicate that both approaches perform well on this task, especially when considering our users' preference for high recall and tolerance for low precision. In future research, we hope to explore the use of multi-lingual embeddings and other machine-learning-based methods on this benchmark and construct similar benchmarks for other language pairs that share a common historical and linguistic background. The benchmark and full results are provided as an electronic supplement to this work.

**Supplementary information** The online version contains supplementary material available at <https://doi.org/10.1007/s10579-025-09812-9>.

**Author contributions** All authors contributed to conceptualizing the work. TS wrote the main manuscript text, wrote the evaluation software and performed the evaluation experiments. SR and MZ created the benchmark dataset, contributed to developing the transliteration mechanism, analyzed the results of the evaluation experiments, and contributed to writing the main text. MRF devised, developed, and described the phoneme-based matching method. JB and KH reviewed the work and consulted on the methodology.

**Funding** Open access funding provided by Aalborg University.

**Data availability** Data is provided in supplementary data files.

## Declarations

**Competing interests** The authors declare no competing interests.

**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

## References

- Ardanuy, M. C., & Sporleder, C. (2017). Toponym disambiguation in historical documents using semantic and geographic features. In: *Proceedings of the 2nd International Conference on Digital Access to Textual Cultural Heritage*. DATeCH2017, (pp. 175–180). Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3078081.3078099>
- Bast, H., & Buchhold, B. (2017). QLever: A query engine for efficient SPARQL+Text search. In: *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. CIKM '17, (pp. 647–656). Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3132847.3132921>
- Benjamin, Asher, A., Zunz, L., & Lebrecht, F. *The Itinerary of Rabbi Benjamin of Tudela*. A. Asher & co.
- Bharadwaj, A., Mortensen, D., Dyer, C., & Carbonell, J. (2016). Phonologically aware neural model for named entity recognition in low resource transfer settings. In: Su, J., Duh, K., Carreras, X. (eds.) *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, (pp. 1462–1472). Association for Computational Linguistics, Austin, Texas. <https://doi.org/10.18653/v1/D16-1153><https://aclanthology.org/D16-1153> Accessed 2024-12-19.
- Carlson, Thomas A. Historical index of the medieval middle east (HIMME). <https://medievalmideast.org/index.html> Accessed: 01/01/2024.
- Catford, J. C., & Esling, J. H. (2006). Articulatory phonetics. *Encyclopedia of Language and Linguistics*, 9, 425–442.
- Christen, P. (2008). Febrl: An open source data cleaning, deduplication and record linkage system with a graphical user interface. In: *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, (pp. 1065–1068).
- Cornu, G. (1983). *Atlas Du Monde Arabo-Islamique a l'Epoque Classique: IXe-Xe Siècles (The Arab-Islamic World and Classic Europe Atlas: 9th-10th Centuries)*. Brill.
- Dolgopolsky, A. B. (1986). A probabilistic hypothesis concerning the oldest relationships among the language families of northern Eurasia. *Typology, relationship and time: A collection of papers on language change and relationship by soviet linguists*, 27–50.
- Gibb, H. A. R. (1998). *The Encyclopaedia of Islam*. Brill Archive.
- Gong, R., & Chan, T. K. (2006). Syllable alignment: A novel model for phonetic string search. *IEICE Transactions on Information and Systems*, 89(1), 332–339.
- Grossner, K., & Mostern, R. (2021). Linked Places in World Historical Gazetteer. In: *Proceedings of the 5th ACM SIGSPATIAL International Workshop on Geospatial Humanities*. GeoHumanities '21, (pp. 40–43). Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3486187.3490203>
- Hastings, J. (2008). Automated conflation of digital gazetteer data. *International Journal of Geographical Information Science*, 22(10), 1109–1127.
- Jaro, M. A. (1989). Advances in record-linkage methodology as applied to matching the 1985 census of Tampa, Florida. *Journal of the American Statistical association*, 84, 414–420.
- Joshi, T., Joy, J., Kellner, T., Khurana, U., Kumaran, A., & Sengar, V. (2008). Crosslingual location search. In: *Proceedings of the 31st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '08, (pp. 211–218). ACM, New York, NY, USA. <https://doi.org/10.1145/1390334.1390372>

- Keskustalo, H., Pirkola, A., Visala, K., Leppänen, E., & Järvelin, K. (2003). Non-adjacent digrams improve matching of cross-lingual spelling variants. In: *String Processing and Information Retrieval: 10th International Symposium, SPIRE 2003, Manaus, Brazil*, October 8–10, 2003. Proceedings 10, (pp. 252–265). Springer.
- Klementiev, A., & Roth, D. (2006). Weakly supervised named entity transliteration and discovery from multilingual comparable corpora. In: Calzolari, N., Cardie, C., Isabelle, P. (eds.) *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the Association for Computational Linguistics*, (pp. 817–824). Association for Computational Linguistics, Sydney, Australia. <https://doi.org/10.3115/1220175.1220278https://aclanthology.org/P06-1103> Accessed 2024-12-19.
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. In: *Soviet Physics Doklady*, (vol. 10, pp. 707–710). Soviet Union.
- MEHDIE: The MEHDIE toponym matching tool. <https://tool.mehdie.org/> Accessed: 01/01/2024.
- MEHDIE Project: The MEHDIE Transliteration Service and Python Package. <https://pypi.org/project/translit-me/> Accessed: 01/01/2024.
- MEHDIE Project: The translit-me source code repository. <https://gitlab.com/m8417/hebrew-transliteration-service> Accessed: 01/01/2024.
- Martins, B. (2011). A supervised machine learning approach for duplicate detection over gazetteer records. In: *International Conference on GeoSpatial Semantics*, (pp. 34–51). Springer.
- Maxim Romanov: al-Turayya Project. <https://althurayya.github.io/> Accessed: 01/01/2024.
- Mortensen, D. R., Littell, P., Bharadwaj, A., Goyal, K., Dyer, C., & Levin, L. (2016). Panphon: A resource for mapping IPA segments to articulatory feature vectors. In: *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, (pp. 3475–3484).
- National Geospatial-Intelligence Agency: Geographic Names Server. <https://geonames.nga.mil/geonames/GNSHome/index.html> Accessed: 01/01/2024.
- Novak, J. R., Minematsu, N., & Hirose, K. (2016). Phonetisaurus: Exploring grapheme-to-phoneme conversion with joint n-gram models in the WFST framework. *Natural Language Engineering*, 22(6), 907–938.
- Papadakis, G., Skoutas, D., Thanos, E., & Palpanas, T. (2020). Blocking and filtering techniques for entity resolution: A survey. *ACM Computing Surveys (CSUR)*, 53(2), 1–42.
- Pleiades Team: Pleiades. <https://pleiades.stoa.org/> Accessed: 01/01/2024.
- Recchia, G., & Louwerse, M. (2013). A comparison of string similarity measures for toponym matching. In: *Proceedings of The First ACM SIGSPATIAL International Workshop on Computational Models of Place*. COMP '13, (pp. 54–61). Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2534848.2534850>
- Santos, R., Murrieta-Flores, P., Calado, P., & Martins, B. (2018). Toponym matching through deep neural networks. *International Journal of Geographical Information Science*, 32(2), 324–348.
- Santos, R., Murrieta-Flores, P., & Martins, B. (2018). Learning to combine multiple string similarity metrics for effective toponym matching. *International Journal of Digital Earth*, 11(9), 913–938.
- Smith, T. F., Waterman, M. S., et al. (1981). Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1), 195–197.
- Sun, K., Zhu, Y., & Song, J. (2019). Progress and challenges on entity alignment of geographic knowledge bases. *ISPRS International Journal of Geo-Information*. <https://doi.org/10.3390/ijgi8020077>
- Sun, Z., Hu, W., & Li, C. (2017). Cross-lingual entity alignment via joint attribute-preserving embedding. In C. d'Amato, M. Fernandez, V. Tamma, F. Lecue, P. Cudré-Mauroux, J. Sequeda, C. Lange, & J. Heflin (Eds.), *The Semantic Web - ISWC 2017* (pp. 628–644). Springer.
- TravelLab: Benjamin of Tudela. (2024). Accessed: March 1st, (n.d.). <https://teipublisher.info/exist/apps/TravelLab/Benjamin%20of%20Tudela.xml>
- Tucci, M., & Giordano, A. (2011). Positional accuracy, positional uncertainty, and feature change detection in historical maps: Results of an experiment. *Computers, Environment and Urban Systems*, 35(6), 452–463. <https://doi.org/10.1016/j.compenvurbsys.2011.05.004>
- Wehr, H. (1979). *A dictionary of modern written Arabic*. Otto Harrassowitz Verlag.
- Yāqūt, al-Rūmī al-Hamawī. (1977). *Kitāb Mu'jam al-Buldān* (The Countries Dictionary Book). Dār Ṣādir, Beirut. Original work published in the 13th Century. 5 vols.
- Zhang, R., Trisedya, B. D., Li, M., Jiang, Y., & Qi, J. (2022). A benchmark and comprehensive survey on knowledge graph entity alignment via representation learning. *The VLDB Journal*, 31(5), 1143–1168.



Zhang, Z., Liu, H., Chen, J., Chen, X., Liu, B., Xiang, Y., & Zheng, Y. (2020). An industry evaluation of embedding-based entity alignment. In: *Proceedings of the 28th International Conference on Computational Linguistics: Industry Track*, (pp. 179–189).

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

## Authors and Affiliations

**Tomer Sagi<sup>1</sup> · Moran Zaga<sup>2</sup> · Sinai Rusinek<sup>2</sup> · Marcell R. Fekete<sup>1</sup> · Johannes Bjerva<sup>1</sup> · Katja Hose<sup>1,3</sup>**

✉ Tomer Sagi  
tsagi@cs.aau.dk

Moran Zaga  
mzaga@staff.haifa.ac.il

Sinai Rusinek  
sinai.rusinek@gmail.com

Marcell R. Fekete  
mrfe@cs.aau.dk

Johannes Bjerva  
jbjerva@cs.aau.dk

Katja Hose  
katja.hose@tuwien.ac.at

<sup>1</sup> Department of Computer Science, Aalborg University, Aalborg, Denmark

<sup>2</sup> e-Lijah Lab, University of Haifa, Haifa, Israel

<sup>3</sup> Institute of Logic and Computation, TU Wien, Vienna, Austria